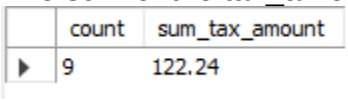
Applied Database Technologies
Lab 4: Summary queries
Luke Colehower

Chapter 6: How to code summary queries

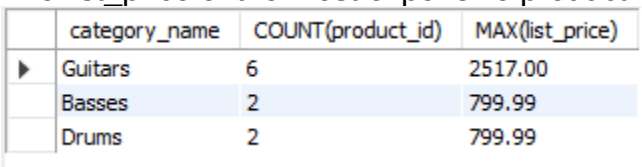
1. Write a SELECT statement that returns these columns:
    - The count of the number of orders in the orders table
    - The sum of the tax_amount columns in the orders table



count	sum_tax_amount
9	122.24

```
SELECT COUNT(order_id) AS count, SUM(tax_amount) AS
sum_tax_amount
FROM orders;
```

2. Write a SELECT statement that returns one row for each category that has products with these columns:
    - The category_name column from the categories table
    - The count of the products in the products table
    - The list_price of the most expensive product in the products table.



category_name	COUNT(product_id)	MAX(list_price)
Guitars	6	2517.00
Basses	2	799.99
Drums	2	799.99

```
SELECT category_name, COUNT(product_id), MAX(list_price)
FROM categories c
JOIN products p USING(category_id)
GROUP BY category_name;
```

3. Write a SELECT statement that returns one row for each customer that has orders with these columns:
    - The email_address column from the customers table
    - The sum of the item_price in the order_items table multiplied by the quantity in the order_items column
    - The sum of the discount_amount column in the order_items table multiplied by the quantity in the order_items table

Sort the result set by the item_price total for each customer in descending sequence.

email_address	total_cost	total_discount
allan.sherwood@yahoo.com	4131.00	1830.39
christineb@solarone.com	2398.00	719.40
frankwilson@sbcglobal.net	2198.98	659.70
david.goldstein@hotmail.com	998.00	209.70
gary_hernandez@yahoo.com	799.99	120.00
barryz@gmail.com	489.99	186.20
erinv@gmail.com	299.00	0.00

```
SELECT email_address , SUM((item_price )*quantity) AS total_cost,

SUM((discount_amount) * quantity) AS total_discount

FROM customers c

JOIN orders o USING(customer_id)

JOIN order_items USING(order_id)

GROUP BY email_address

ORDER BY  SUM((item_price )*quantity) DESC;
```

4. Write a SELECT statement that returns one row for each customer that has orders with these columns:
   - The email_address column from the customers table
   - A count of the number of orders
   - The total amount for each order item (*Hint: First, subtract the discount amount from the price. Then, multiply by the quantity*.)

   Return only those rows where the customer has more than 1 order.

   Sort the result set by total amount in descending order.

email_address	num_orders	total_amount
allan.sherwood@yahoo.com	2	2300.61
david.goldstein@hotmail.com	2	788.30

```
SELECT email_address, COUNT(order_id) AS num_orders,

SUM((item_price-discount_amount)* quantity) AS total_amount

FROM customers c

JOIN orders o USING (customer_id)
```

```
JOIN order_items oi USING(order_id)

GROUP BY customer_id, email_address

HAVING COUNT(order_id) > 1

ORDER BY SUM((item_price-discount_amount)* quantity) DESC ;
```

5. Modify the solution to exercise 4 so it only counts and totals order_items that have an item_price value that's greater than 400.



email_address	num_orders	total_amount
allan.sherwood@yahoo.com	2	2300.61

```
SELECT email_address, COUNT(order_id) AS num_orders,

SUM((item_price-discount_amount)* quantity) AS total_amount

FROM customers c

JOIN orders o USING (customer_id)

JOIN order_items oi USING(order_id)

WHERE item_price > 400

GROUP BY customer_id, email_address

HAVING COUNT(order_id) > 1

ORDER BY SUM((item_price-discount_amount)* quantity) DESC ;
```

6. Write a SELECT statement that answers this question: What is the total amount ordered for each product? Return these columns:
   - The product_name column from the products table
   - The total amount for each product in the order_items table (*Hint: You can calculate the total amount by subtracting the discount_amount from the item_price and then multiplying it by the quantity*)

   Use the WITH ROLLUP operator to include a row that gives the grand total.

product_name	total_amount
Fender Precision	559.99
Fender Stratocaster	978.60
Gibson Les Paul	2517.90
Gibson SG	1208.16
Ludwig 5-piece Drum Set with Cymbals	489.99
Rodriguez Caballero 11	253.15
Tama 5-Piece Drum Set with Cymbals	679.99
Washburn D10S	598.00
Yamaha FG700S	303.79
NULL	7589.57

```
SELECT product_name, SUM((item_price - discount_amount) * quantity) AS
Total_amount

FROM products p

JOIN order_items oi USING (product_id)

GROUP BY product_name WITH ROLLUP;
```

7. Write a SELECT statement that answers this question: Which customers have ordered more than one product? Return these columns:
   - The email_address column from the customers table
   - The count of distinct products from the customer's orders

   Sort the result set in descending order by the email_address column.

email_address	count_products
frankwilson@sbcglobal.net	3
david.goldstein@hotmail.com	2
allan.sherwood@yahoo.com	3

```
SELECT email_address, COUNT(DISTINCT product_id) AS count_products

FROM customers c

JOIN orders o USING(customer_id)

JOIN order_items oi USING(order_id)

GROUP BY customer_id, email_address

HAVING COUNT(DISTINCT product_id) > 1

ORDER BY email_address DESC;
```

8. Write a SELECT statement that answers this question: What is the total quantity purchased for each product within each category? Return these columns:
    - The category_name column from the category table
    - The product_name column from the products table
    - The total quantity purchased for each product with orders in the order_items table

Use the WITH ROLLUP operator to include rows that give a summary for each category_name as well as a row that gives the grand total.

Use the IF and GROUPING functions to replace null values in the category_name and product_name columns with literal values if they're for summary rows.

category_name	product_name	SUM(quantity)
Basses	Fender Precision	1
Basses	**Category Ordered Products Total**	1
Drums	Ludwig 5-piece Drum Set with Cymbals	1
Drums	Tama 5-Piece Drum Set with Cymbals	1
Drums	**Category Ordered Products Total**	2
Guitars	Fender Stratocaster	2
Guitars	Gibson Les Paul	3
Guitars	Gibson SG	1
Guitars	Rodriguez Caballero 11	1
Guitars	Washburn D10S	2
Guitars	Yamaha FG700S	1
Guitars	**Category Ordered Products Total**	10
Total Ordered Products	NULL	13

```
SELECT IF(GROUPING(category_name) = 1, '**Total Ordered Products',
category_name) AS category_name,

IF(GROUPING(product_name) = 1, '**Category Ordered Products Total'
,product_name) AS product_name,

 SUM(quantity)

FROM categories c

LEFT JOIN products p USING(category_id)

RIGHT JOIN order_items oi USING(product_id)

GROUP BY category_name, product_name WITH ROLLUP;
```

Extra Credit:

9. Write a SELECT statement that uses an aggregate window function to get the total amount of each order. Return these columns:
    - The order_id from the order_items table
    - The total amount for each order item in the order_items table (*Hint: You can calculate the total amount by subtracting the discount amount from the item price and then multiplying it by the quantity*)
    - The total quantity purchased for each product with orders in the order_items table

    Sort the result set in ascending sequence by the order_id column.

order_id	total_amount	total_quantity
1	839.30	1
2	303.79	1
3	1461.31	2
3	1461.31	2
4	1678.60	2
5	299.00	1
6	299.00	1
7	1539.28	3
7	1539.28	3
7	1539.28	3
8	679.99	1
9	489.30	1

```
SELECT order_id, SUM((item_price- discount_amount)* quantity)

OVER(PARTITION BY order_id) AS total_amount,

SUM(quantity) OVER(PARTITION BY order_id) AS total_quantity

FROM order_items oi

ORDER BY order_id;
```

10. Modify the solution to exercise 9 so the column that contains the total amount for each order contains a cumulative total by item amount.

    Add another column to the SELECT statement that uses an aggregate window function to get the average item amount for each order.

    Modify the SELECT statement so it uses a named window for the two aggregate functions.

order_id	total_amount	total_quantity	avg_amount
1	839.30	1	839.300000
2	1143.09	2	571.545000
3	2604.40	4	651.100000
3	2604.40	4	651.100000
4	4283.00	6	856.600000
5	4582.00	7	763.666667
6	4881.00	8	697.285714
7	6420.28	11	642.028000
7	6420.28	11	642.028000
7	6420.28	11	642.028000
8	7100.27	12	645.479091
9	7589.57	13	632.464167

```
SELECT order_id, SUM((item_price- discount_amount)* quantity)

OVER OD AS total_amount,

SUM(quantity) OVER OD AS total_quantity,

AVG((item_price-discount_amount)*quantity) OVER OD AS Average_amount

FROM order_items oi

WINDOW OD AS(PARTITION BY order_id ORDER BY item_id

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

ORDER BY order_i
```

11. Write a SELECT statement that uses aggregate window functions to calculate the order total for each customer and the order total for each customer by date. Return these columns:
    - customer_id
    - order_date
    - The total amount for each order item in the order_items table
    - The sum of the order totals for each customer
    - The sum of the order totals for each customer by date *(Hint: You can create a peer group to get these values)*

customer_id	order_date	order_total	total_customer_order	total_customer_order_by_date
1	2018-03-28 09:40:28	839.30	2300.61	839.30
1	2018-03-29 09:44:58	1461.31	2300.61	1461.31
2	2018-03-28 11:23:20	303.79	303.79	303.79
3	2018-03-30 15:22:31	1678.60	1678.60	1678.60
4	2018-03-31 05:43:11	299.00	788.30	299.00
4	2018-04-03 12:22:31	489.30	788.30	489.30
5	2018-03-31 18:37:22	299.00	299.00	299.00
6	2018-04-01 23:11:12	1539.28	1539.28	1539.28
7	2018-04-02 11:26:38	679.99	679.99	679.99

```
SELECT customer_id, order_date,

SUM((item_price-discount_amount)*quantity) AS order_total,

SUM(SUM((item_price-discount_amount)*quantity)) OVER(PARTITION
BY customer_id) AS

total_customer_order,

SUM(SUM((item_price-discount_amount)*quantity)) OVER(PARTITION
BY customer_id, order_date)

AS total_customer_order_by_date

FROM order_items oi

JOIN orders o USING (order_id)

GROUP BY customer_id, order_date, order_id

ORDER BY customer_id, order_date;
```